



SecureVault

Design Report

ENCS4320 Applied Cryptography — Final Project (Term 1253)

Prepared by

Abd-Alrahman Naser — 1230146

Khalid Omari — 1192973

Mohammad Zaid — 1221265

Birzeit University — Faculty of Engineering and Technology

Two users (Layla, Omar) exchange documents through a server that never sees plaintext or keys. An active network attacker cannot read, tamper with, or forge documents undetected. We aim for 128-bit security everywhere; all timings are from one of our laptops (i7-1065G7, Python 3.13). All crypto primitives (SHA-256, HMAC, AES, GCM, P-256, ECDH, ECDSA) are implemented from scratch; the only external cryptographic library used in production code is `argon2-cffi`.

1. Threat model

Three parties exist in this system, and what each one can do decides everything that follows.

Users. Each user runs our client on a machine we assume is honest — we don't defend against malware or a compromised OS. Each user has a password known only to them, and the client never sends it anywhere; everything derived from it stays on the user's own machine except a single public key bundle the server is handed at signup.

The server. Honest-but-curious: it follows the protocol correctly, doesn't delete or fabricate data, but reads and stores everything it's given, and we assume its entire database can end up in an attacker's hands. It stores, per user, only a salt and `auth_pub`; per document, the encrypted SVD1 blob and version; per share, the encrypted SVS1 blob. None of that is enough by itself to read a document, forge a share, or log in as anyone.

The attacker. Controls the network completely: reads, modifies, drops, delays, and replays every message between every client and the server, and can register accounts of their own. We treat "the server has been fully compromised" and "the network attacker" as the same threat, since the server sees exactly what the network attacker would see and nothing more.

Out of scope. Malware or physical access to a user's own machine, coercing a user directly, traffic-analysis of message sizes and timing, and denial of service. We don't claim protection against any of these.

2. Design

2.1 Architecture

Client and server talk over TCP. Every message is one frame: `u32 length || u8 type || body`, with integers big-endian and strings encoded as `u16 length || UTF-8`.

`attacker_proxy.py` is a separate program that sits between client and server and can flip ciphertext bytes, alter metadata, replay a captured request, or substitute a public key, so we can demonstrate live that each of those is rejected rather than just argue that it should be. The server stores accounts, key bundles, encrypted keys, documents, and shares — it can't read files (it never has `FK`), forge anything (it never has a user's signing key), or get passwords cheaply (it only ever holds the salt and `auth_pub`).

2.2 Key hierarchy

Everything traces back to two independent roots: the user's password, and randomness generated at signup that never depends on the password.

- **From the password:** Argon2id turns $(\text{password}, \text{salt})$ into master . HMAC-SHA256 then splits it the same way as $\text{ke} = F(k, 0), \text{km} = F(k, 1)$: $\text{auth_key} = \text{HMAC}(\text{master}, \text{"auth"})$ (used once, to sign the login challenge) and $\text{kek} = \text{HMAC}(\text{master}, \text{"kek"})$ (unlocks the identity private keys). Neither value can do the other's job.
- **Independent of the password:** at signup, the client generates two long-term EC keypairs — a signing keypair (signs documents, shares, and every authenticated request) and an exchange keypair (receives document keys via ECDH). These survive a password change untouched; on a password change we simply re-encrypt the same identity keys under the new kek .
- **Per document:** a fresh AES-256 key FK , generated at upload and never reused for anything else.
- **Per share:** a fresh ephemeral EC keypair for that one ECDH exchange, discarded immediately after it wraps FK for the recipient.

2.3 Byte-level formats

Fixed sizes: doc_id 16 bytes, GCM nonce 12 bytes, GCM tag 16 bytes, document hash 32 bytes, ephemeral public key 65 bytes (uncompressed SEC1 P-256 point), wrapped FK 32 bytes, ECDSA signature 64 bytes ($r||s$, no DER).

Object	Fields
Document SVD1	header: magic(4), doc_id (16), owner(str), version u32, created_ms u64 — this header is the AES-GCM AAD — then nonce(12), ciphertext, tag(16)
Encrypted payload	filename(str), mime(str), content, content_sig(64)
Share SVS1	magic(4), doc_id (16), version u32, doc_hash (32), sender(str), recipient(str), issued_ms u64, eph_pub(65), nonce(12), wrapped_fk(32), tag(16), share_sig(64)

doc_id , owner, version, and timestamp stay readable and go in the AAD. The server can read them but can't change them, and neither can the network attacker — any change breaks the GCM tag.

2.4 Life of a document

Layla encrypts a file under a fresh FK with AES-GCM, signs the file content with her signing key (so a third party can later verify authorship with only her public key), and uploads the SVD1 blob at version 1. She then self-shares the document — wrapping FK for herself through ECDH — so she can download her own upload later; the server relays this without ever seeing FK . To share with Omar, she looks up his public keys (through TOFU, Sec. 3.4), wraps FK again for him with a fresh ephemeral ECDH exchange, and sends only that wrapped copy. Omar downloads the ciphertext and

Layla's `SVS1` share blob, checks the fingerprint on her key, verifies her signature over the content before trusting it, unwraps `FK` with his own exchange key, and only then runs GCM decryption, which itself checks the tag before releasing any plaintext. A failure at any single step — wrong signature, changed metadata, wrong key, bad tag — stops the whole chain right there.

3. Justification

3.1 Passwords

A bare SHA-256 is too fast: a GPU does billions of hashes per second. A slow hash alone isn't enough either, because a GPU runs thousands of guesses in parallel — a memory-hard hash stops that, since each GPU core has limited memory. PBKDF2 isn't memory-hard, bcrypt isn't either, and Argon2id is the recommended variant. So we picked Argon2id.

Work factor: $t=2$, $m=128$ MiB, $p=1$, takes 235 ms on our laptop, which is inside the recommended 0.1–0.5 s range. The parameters are stored in the user record so we can raise them later. We also tried a few other settings with `argon2-cffi` ($p=1$, best of 5 runs):

t	m	time
1	128 MiB	116 ms
2	64 MiB	97 ms
2	128 MiB	195 ms
3	128 MiB	269 ms
2	256 MiB	372 ms

($t=2$ / 128 MiB gave 235 ms the first time and 195 ms when we ran it again; we kept 235 ms for the math below, since a slightly conservative estimate is the safer one to publish.)

Attack estimate for an offline guessing attack. 8 characters from a-z A-Z 0-9 gives $62^8 \approx 2.2 \times 10^{14}$ passwords.

- Bare SHA-256, no salt, 10^{10} hashes/s: $2.2 \times 10^{14} / 10^{10} \approx 22,000$ s ≈ 6 hours, and that breaks all accounts at once.
- Our Argon2id: one core on our laptop does $1 / 0.235 \approx 4$ guesses/s. We don't have a GPU to test, so we assumed an attacker with 1,000 cores as fast as ours (4,000 guesses/s): $2.2 \times 10^{14} / 4,000 \approx 5.5 \times 10^{10}$ s $\approx 1,700$ years for one account. In practice it should be even slower, because every guess needs 128 MiB and a GPU core doesn't have that much memory.
- Common passwords are still a problem. A top-10,000 list takes $10,000 \times 0.235$ s ≈ 40 minutes per account on one core. The salt doesn't help with guessing, only with precomputation.

Salt: `os.urandom(16)`, 16 random bytes, new at sign-up and at every password change, stored in the clear. It isn't secret because its job is uniqueness: two users with the same password get different records, and the attacker has to go after one account at a time.

Second use of the password: Argon2id runs on the client and gives `master`. The server only ever keeps the salt and `auth_pub` — nothing that lets it sign a request, decrypt a document, or stand in for the password. At login it sends a random challenge and the client signs it. If the password changes, we re-encrypt the same identity keys with the new `kek`; they don't come from the password, so contacts and existing shares still work.

Unknown users: if the server said "no such user" or didn't send a salt, anyone could check which accounts exist. So for an unknown username it derives a fake salt as `HMAC(server_secret, username)`. The same name always gets the same fake salt, so asking twice doesn't give it away, and the reply is `LOGIN_FAILED` either way.

3.2 Document contents

AES-256-GCM, written by us. GCM is CTR + GHASH in encrypt-then-MAC order, so a changed ciphertext is rejected before anything is decrypted. If the tag is wrong, decryption returns `⊥` and never the plaintext.

- CBC + HMAC done by hand: if we decrypted before checking, we'd have a padding oracle. If we did build it ourselves, the safe order is encrypt-then-MAC with two separate keys.
- CCM: the MAC is over the plaintext, so the receiver has to decrypt before checking. There's no padding oracle in CCM since CTR has no padding. We still went with GCM because it checks before decrypting and uses about one cipher call per block, not two.

Keys per document: one random file key `FK`, plus one wrapped copy of `FK` for each person with access, the owner included.

Nonces: reusing a (key, nonce) pair in GCM leaks the XOR of the plaintexts and the GHASH key `H` — our `weak_build_demo.py` shows exactly this, recovering an entire second file from two ciphertexts that share a (key, nonce) pair. We avoid the whole failure class at the root: every GCM key in our system encrypts exactly one message — `FK` is new for every upload, the wrap key is new for every share, and `kek` is new after every password change — so the nonce can safely be a fixed all-zero value; there is nothing to reuse it against. `weak_build_demo.py` also runs the same scheme with fresh random nonces per call to show the identical attack recovers nothing once the (key, nonce) pair is no longer repeated. Files over 64 GiB are refused.

Header: `doc_id`, owner, version, and timestamp stay readable and go in the AAD. The server can read them but can't change them, and neither can the network attacker.

3.3 Public-key crypto

With symmetric keys only, we'd need a key for every pair of users ($n(n-1)/2$) or a KDC, and our server can't be trusted as a KDC. Diffie-Hellman lets two people who never met agree on a key, and that key goes into AES-GCM. So we use ECDH to send `FK` to another user, and ECDSA to prove who wrote a file.

We picked elliptic curves over the multiplicative group of integers mod p , because that group has much faster special attacks and needs bigger numbers. On a curve the best attacks are square-root attacks, so a group of about 2^{256} gives 2^{128} security. That's P-256, and we use the same curve for both jobs so we only write it once.

At 128-bit security, ECC needs 256 bits where RSA/DH needs about 3072 bits, so keys are about $12\times$ smaller. For speed we ran `openssl speed -seconds 3 ecdhp256 ecdsap256 rsa3072 ffdh3072` (OpenSSL 3.2.2) on our laptop:

Operation	ops/s
ECDH P-256	12,847
DH-3072	767
ECDSA P-256 sign	22,282
RSA-3072 sign	1,145

That's about $17\times$ faster for key agreement and about $19\times$ faster for signing. These numbers are from OpenSSL's C code, not our Python code, but we only wanted the ratio between ECC and RSA/DH.

SHA-256 also gives 128-bit collision security, so all parts match. AES-256 doesn't make the system 256-bit.

3.4 Trusting a public key

If the server gives Layla its own key instead of Omar's, it becomes a man-in-the-middle.

We use trust-on-first-use plus fingerprints. The first time Layla gets Omar's key, her client saves it as UNVERIFIED. They compare the fingerprint (SHA-256 of his key bundle) in person or on a call, and then she marks it VERIFIED. If the server later sends a different key for the same username, the client blocks sharing until they compare again — there is no "trust anyway" path once a mismatch is detected.

No CA on the server: the server is the thing we don't trust, and one central authority is a single point of failure. Weak spot: if users never compare, a key swapped on the very first download isn't noticed.

3.5 Signatures vs MACs

- MAC (GCM tag, HMAC): integrity between people who already share a key.
- Signature (ECDSA): document content, shares, and every authenticated request.

Only a signature gives non-repudiation. With a MAC both sides have the same key, so a judge can't tell if Layla or Omar made the tag — making the tag longer doesn't change this, because both of them still have the key. Only Layla has her private key, so only she could have signed.

We sign the SHA-256 of the file content itself. Omar can then give a third party the file, the signature, and Layla's public key, and they can check it without any secret key at all. Every share is separately signed over its sender, recipient, `doc_id`, version, and timestamp, so a share cannot be replayed onto a different recipient or a different document.

3.6 Architecture and freshness

Every sensitive request (upload, share, download, password change) is signed with the caller's own signing key and carries a fresh `request_nonce`; the server checks the signature against the public key it was handed at signup and rejects any nonce it has already accepted, so a captured request can never be replayed as-is. Login challenges are single-use and expire after 60 seconds if never consumed.

Document updates are additionally version-checked: the server only accepts a re-upload whose version is exactly `current + 1`, and rejects anything else as stale — this is what stops an attacker from replaying an old, validly-signed version of a document over a newer one.

On failure: nothing is saved or shown, and nothing about the failure is sent back to the server. Tags are checked before decryption and compared in constant time. Same error text isn't enough on its own, since the oracle can hide in the control flow — we check the tag first, so that path isn't there in our code.

3.7 Library use

Everything is ours — SHA-256, HMAC-SHA256, AES (block cipher plus our own CTR + GHASH for GCM), P-256 point arithmetic, ECDSA, ECDH, and HKDF — except Argon2id. A pure-Python Argon2 with 128 MiB would be far too slow, so we'd have to cut the memory and lose the reason we chose it.

We didn't just assert correctness. Every primitive is checked against published test vectors where they exist, and cross-checked in both directions against the `cryptography` library for everything else. All of that lives in `tests/` (198 tests, covering unit, integration, and end-to-end TCP scenarios).

4. Security argument

Property	Meaning here	Mechanism
Confidentiality	Nobody but the intended users reads a document, server included.	AES-256-GCM under a per-document key FK that's never reused; FK itself only ever travels wrapped by ECDH for one specific recipient (Sec. 3.2, 3.3).
Integrity	Any alteration is caught before use.	The GCM tag is checked, in constant time, before any plaintext is released (Sec. 3.2, 3.6).
Data-origin authentication	The recipient learns who produced the document.	The owner's ECDSA signature over the file content, verified against a key the recipient has fingerprint-checked (Sec. 3.4, 3.5).
Non-repudiation	The recipient can prove	The same ECDSA signature: only

	authorship to a third party.	the owner's private key could have produced it, and a third party can verify it with only the public key (Sec. 3.5).
Credential protection	A stolen database doesn't yield passwords at scale.	Argon2id, memory-hard and tuned to ~235 ms/guess; the server never stores anything password-equivalent, only the public <code>auth_pub</code> (Sec. 3.1).
Freshness of requests	Replayed requests are rejected.	Every accepted <code>request_nonce</code> is remembered server-side and never accepted twice; login challenges are single-use with a 60 s TTL (Sec. 3.6).
Freshness of documents	An old version can't be replayed over a newer one.	The server only accepts an update whose version is exactly <code>current + 1</code> (Sec. 3.6).
Metadata binding	Nothing the server sees in the clear can be altered or moved to another document undetected.	<code>doc_id</code> , owner, version, and timestamp are the AES-GCM AAD, so any change breaks the GCM tag (Sec. 2.3, 3.2).

These interact rather than stack independently: metadata binding is what stops an attacker from taking a validly-signed, validly-encrypted document and re-labeling it as a different one; freshness is what stops them from replaying a validly-signed request or an old document version instead of forging a new one; and non-repudiation only holds because we chose a signature over a MAC in the first place. Getting the order right inside GCM (encrypt-then-MAC) is what keeps confidentiality and integrity from interfering with each other.

5. Limitations

- **Common passwords still fall.** About 40 minutes per account for a top-10,000 password list, salt or no salt. Fixing this needs a dictionary check at sign-up, which we didn't build.
- **TOFU's first contact is unverified by construction.** If two users never compare fingerprints out of band, a key swapped on the very first download or share goes unnoticed. Fixing this needs an out-of-band channel we can't assume exists (a phone call, a QR code scanned in person).
- **A forgotten password means the files are gone.** The identity keys are only recoverable through `kek`, which only exists as a function of the real password. A recovery path would need a second trusted party or an escrow key, both of which weaken the "only the user can decrypt" guarantee.
- **The server sees traffic metadata.** Who shares with whom, file sizes, and request timing are all visible even though contents aren't. Hiding this needs padding and dummy traffic, which we didn't implement and which cost real bandwidth.
- **The TCP connection itself isn't encrypted.** An attacker can already see every field name and size; wrapping the socket in TLS would hide that shape without changing anything above it.

- **No forward secrecy for old shares.** If Omar's exchange private key is stolen later, every document key ever wrapped for him can be unwrapped, because the wrapping is static ECDH, not a ratchet. Fixing this needs per-session ephemeral-ephemeral exchange and forgetting old keys, which is a meaningfully bigger protocol.
- **Python isn't constant-time by construction.** Our own AES, SHA-256, and EC arithmetic are ordinary Python; branch prediction and cache timing on the underlying hardware could leak information a hand-tuned C or hardware implementation wouldn't. We only made the comparisons that matter (tag checks, signature verification) explicitly constant-time; the arithmetic underneath isn't guaranteed to be.
- **Quantum computers break elliptic-curve cryptography.** Shor's algorithm solves the discrete-log problem P-256 relies on. Nothing here is post-quantum; a real fix means replacing ECDH/ECDSA with a lattice-based KEM and signature scheme, which is a different project.

We're not claiming unconditional security anywhere above — these are the specific things this design doesn't stop, and roughly what it would take to stop each one.

6. AI usage

The use of AI tools in this project complies with the AI Usage Statement outlined in Section 14 of the project guidelines.